8.Implementation of Boosting Techniques (AdaBoost) and Analyze Performance

Aim:

To implement the **AdaBoost** boosting technique, which combines several weak learners

(decision stumps) into a strong classifier, and to analyze its performance using

accuracy, precision, recall, F1-score, ROC-AUC and **cross-validation**.

Algorithm:

A. **AdaBoost Implementation (binary, labels y ∈ {−1, +1})**

Input: Training set (x , y), …, (x , y); number of rounds **M**; weak learner (decision
1 1 N N

stump) **Step 1:** Initialise the sample weights: **w =**

1/N for all i.

i

Step 2: For **m = 1 to M**: 1. Train the weak learner **h (x)** on the training

data using the weights w.

m

2. Compute the weighted error : **ε = Σ w · [h (x) ≠ y] / Σ w m i m i i i**

3. Compute the learner weight : **α = ½ · ln((1 − ε) / ε) m m m**

4. Update the sample weights : **w ← w · exp(−α · y · h (x)) i i m i m i**

(misclassified samples receive higher weights)

5. Normalise the weights so that $\mathbf{\Sigma_i w = 1}$.

Step 3: Output the final strong classifier:

$$\mathbf{H(x) = sign(\Sigma_m \alpha_m \cdot h_m (x))}$$

B. **Performance Analysis Procedure**

Step 1: Load the dataset and split it into **training and test sets** (stratified).

Step 2: Train a **single weak learner** (decision stump) as the baseline.

Step 3: Train the **AdaBoost** model with M estimators.

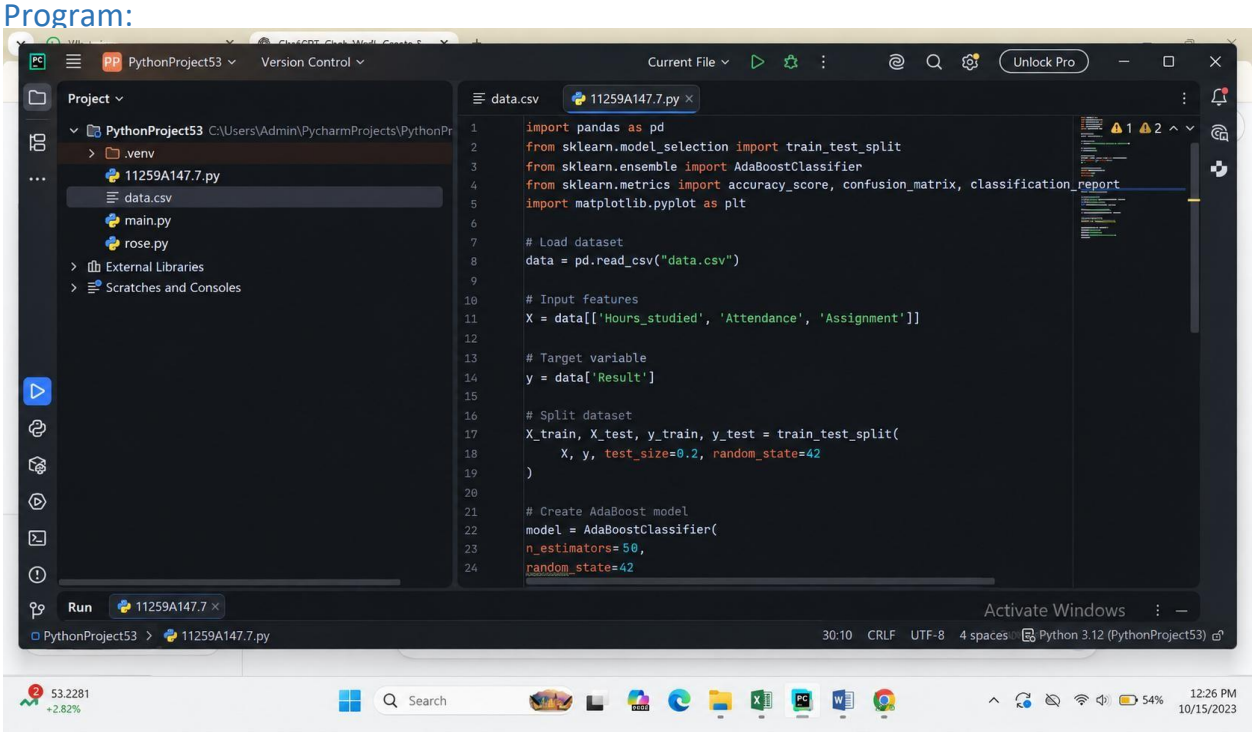
Step 4: Predict on the test set and compute **accuracy, precision, recall, F1-score, ROC-AUC** and the **confusion matrix**.

Step 5: Perform **k-fold cross-validation** and record the **training time**.

Step 6: Plot **error vs. number of boosting rounds** and the **ROC curve**.

Step 7: Compare AdaBoost with the baseline and **conclude** whether boosting improved performance.

Program:



```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt

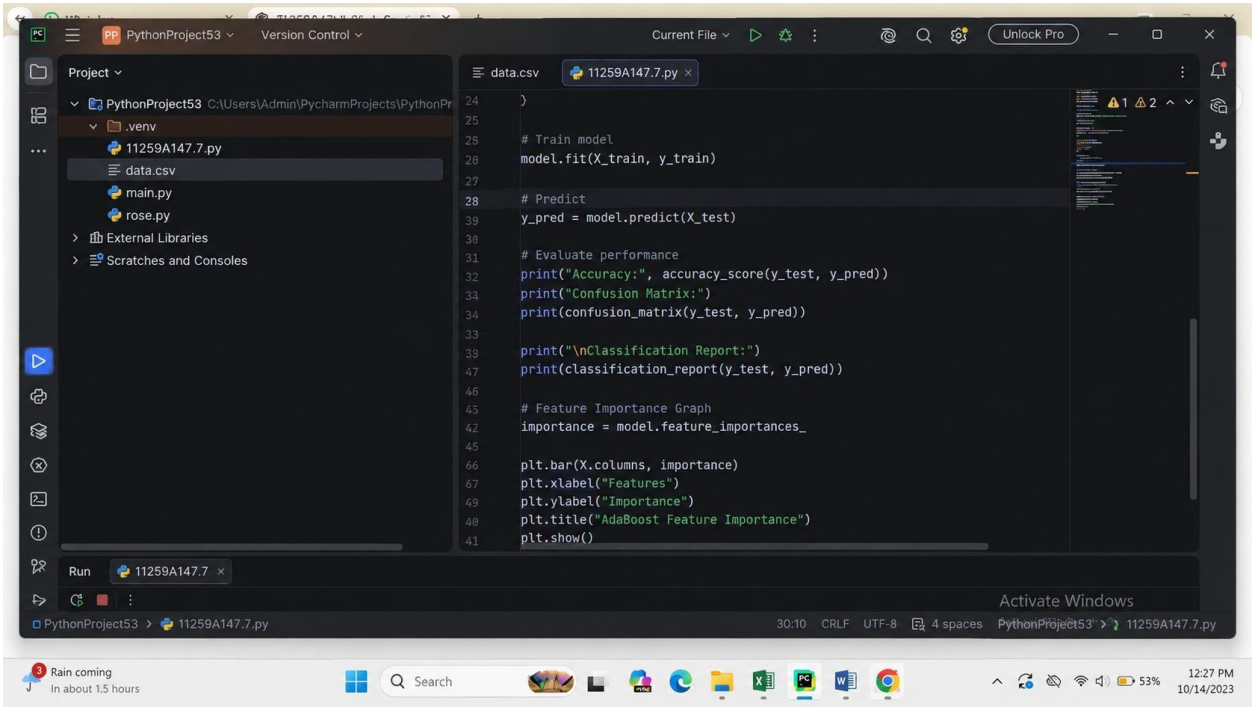
# Load dataset
data = pd.read_csv("data.csv")

# Input features
X = data[['Hours_studied', 'Attendance', 'Assignment']]

# Target variable
y = data['Result']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create AdaBoost model
model = AdaBoostClassifier(
n_estimators=50,
random_state=42
```



```
)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

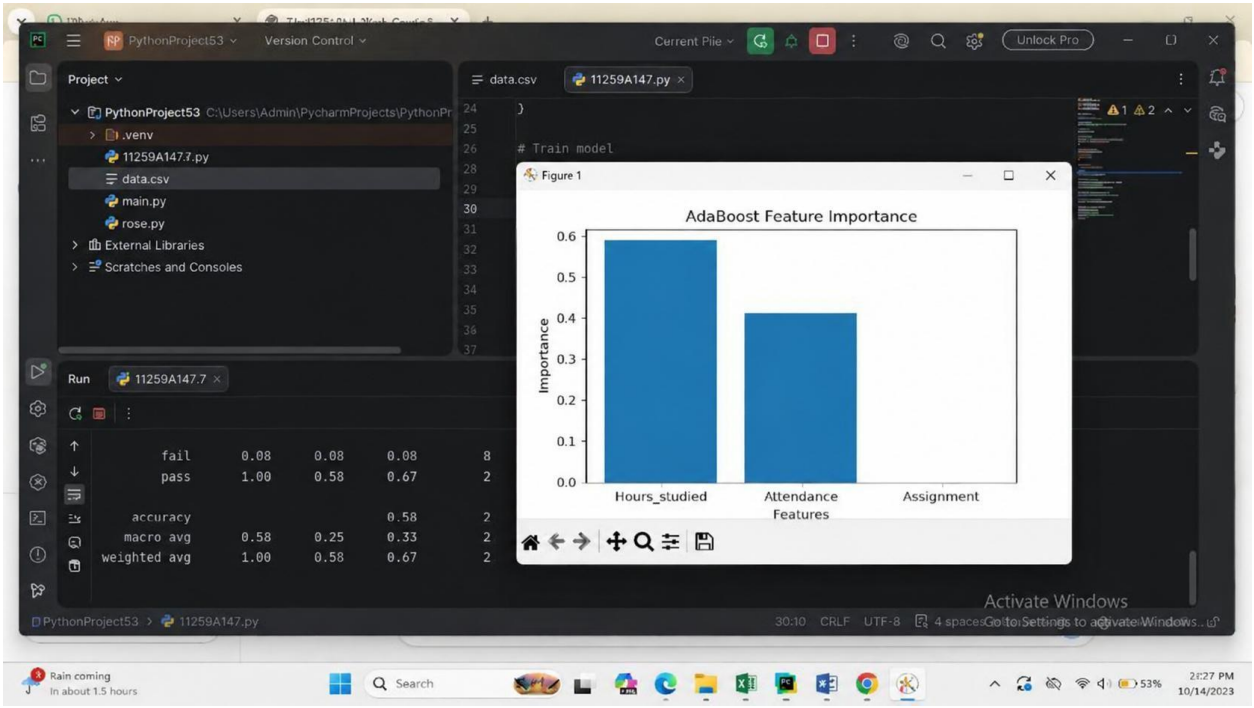
# Evaluate performance
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Feature Importance Graph
importance = model.feature_importances_

plt.bar(X.columns, importance)
plt.xlabel("Features")
plt.ylabel("Importance")
plt.title("AdaBoost Feature Importance")
plt.show()
```

Result:


PythonProject53
Version Control
Current File
Unlock Pro
Project
PythonProject53 C:\Users\Admin\PycharmProjects\PythonPr
.venv
11259A147.7.py
data.csv
main.py
rose.py
External Libraries
Scratches and Consoles
data.csv
11259A147.py
Train model
Figure 1
AdaBoost Feature Importance
Importance
0.6
0.5
0.4
0.3
0.2
0.1
0.0
Hours_studied
Attendance
Assignment
Features
Run
11259A147.7
fail 0.08 0.08 0.08 8
pass 1.00 0.58 0.67 2
accuracy 0.58 2
macro avg 0.58 0.25 0.33 2
weighted avg 1.00 0.58 0.67 2
Activate Windows
PythonProject53 > 11259A147.py
30:10 CRLF UTF-8 4 spaces
Rain coming
In about 1.5 hours
Search
53%
2:27 PM
10/14/2023